Estado	Finalizado
Comenzado	miércoles, 14 de octubre de 2026, 18:19
Completado	miércoles, 14 de octubre de 2026, 18:58
Duración	39 mins 23 segs

Pregunta 1

Finalizado

Sin calificar

Imagina que tienes un sitio web con 100 fotos y un archivo HTML. Si cambias alguna foto, ¿el servidor estático necesita procesar o calcular algo nuevo antes de entregarlos, o simplemente los envía tal como están guardados en el disco? Explica por qué.

Los envía tal como están guardados en el disco porque un servidor estático retorna archivos tal como están almacenados en disco (HTML, CSS, imágenes). Sin procesamiento en tiempo real. Y cuando un cliente (navegador) solicita un recurso, el servidor busca exactamente ese archivo en el disco y lo envía por la red sin modificarle nada.

El servidor estático **los envía tal como están guardados en el disco**, sin procesar ni calcular nada. Esto se debe a que su única función es buscar el archivo solicitado (como un `.png` o un `.html`) y transferir sus bytes por la red. Como no ejecuta código interno ni consulta bases de datos, no realiza cómputos adicionales antes de responder.

Pregunta 2

Finalizado

Sin calificar

Cuando entras a un sitio de comercio electrónico y ves tu carrito de compras con los productos que elegiste, ¿por qué esa página la tiene que fabricar un **servidor de aplicaciones** (dinámico) y no un servidor estático? ¿Qué herramientas utiliza para saber qué productos mostrarte?

Porque un servidor estático solo puede entregar archivos ya terminados mientras que un carrito electrónico tiene que ejecutarse en tiempo real, tiene una logica interna de funcionamiento y debe almacenar la información.

Un servidor dinámico, cuando recibe una petición, no entrega un archivo estático del disco. En su lugar, pone a trabajar a un intérprete o motor de programación (Node.js, Python, PHP, Java), consulta una base de datos (por ejemplo, PostgreSQL o MySQL), combina los datos con una plantilla y fabrica una página HTML totalmente personalizada en ese instante para enviársela al cliente.

Porque el carrito de compras es **contenido dinámico y personalizado** para cada usuario, por lo que no existe como un archivo HTML fijo prefabricado en el disco. El servidor de aplicaciones ejecuta código de programación (en tecnologías como Node.js, Python o PHP) y **consulta una base de datos** para buscar los productos específicos guardados en tu cuenta y armar la página en el instante.

Pregunta 3

Finalizado

Sin calificar

En una empresa, los servidores reales donde se guarda la información no se conectan directamente a Internet por seguridad. ¿Qué papel cumple el **Proxy Inverso** frente a los usuarios de Internet y por qué ayuda a proteger los servidores internos?

Oculta la existencia y direcciones IP reales de los servidores de origen internos.

El Proxy Inverso actúa como una **fachada, escudo o intermediario**. Es la única máquina visible desde Internet; recibe todas las peticiones de los usuarios y las reenvía a los servidores internos. Protege la infraestructura porque **oculta las direcciones IP reales** de los servidores de origen y asume la tarea de verificar y descifrar el tráfico seguro (HTTPS), evitando que atacantes externos accedan directamente a la red privada.

Pregunta 4

Finalizado

Sin calificar

Si un sitio web recibe un millón de visitas en un solo minuto y un solo servidor se caería por la cantidad de tráfico: ¿Cómo ayuda el **Balanceador de Carga** a resolver este problema y qué hace el algoritmo *Round Robin* para repartir a los visitantes?

El balanceador de cargas reparte las peticiones entre los nodos de un cluster, es decir, entre varios servidores. Así el tráfico no recae sobre uno solo.

Round Robin distribuye las peticiones secuencialmente entre todos los nodos del cluster.

En lugar de que un solo servidor reciba todas las visitas, el balanceador las reparte entre todos los nodos, y Round Robin lo hace de forma secuencial.

El Balanceador de Carga resuelve el problema **distribuyendo el tráfico entre un grupo (cluster) de varios servidores**, evitando que uno solo se sobrecargue. El algoritmo **Round Robin** funciona repartiendo las visitas una por una en formato de ciclo o turno (por ejemplo: visita 1 al Servidor A, visita 2 al Servidor B, visita 3 al Servidor C y vuelve a empezar con la visita 4 en el Servidor A).

Pregunta 5

Finalizado

Sin calificar

¿Por qué un servidor de caché guarda en su memoria las páginas o imágenes que los usuarios piden con más frecuencia? ¿En qué beneficia esto al usuario que navega y a la red del servidor original?

El caché guarda lo que más se pide para responder más rápido al usuario y para que el servidor de origen trabaje menos. Porque de esta forma evita recorrer toda la red de área amplia (WAN), responde desde la RAM local.

Guarda esos recursos en su memoria RAM rápida aplicando el **principio de localidad**, ya que es muy probable que muchos usuarios pidan exactamente la misma información repetidamente.

- **Beneficio para el usuario:** Recibe la respuesta de forma casi instantánea (menor tiempo de espera o latencia).
- **Beneficio para el servidor original:** Reduce drásticamente la cantidad de trabajo y tráfico que llega al servidor principal, evitando que colapse por peticiones repetitivas.

Pregunta 6

Finalizado

Sin calificar

Cuando un usuario en Argentina accede a un sitio web alojado en un servidor de origen ubicado en Alemania, ¿cómo logra una CDN (Content Delivery Network) entregar el contenido casi de forma instantánea? Explica la función de los Nodos Edge y cómo opera el Enrutamiento Anycast para minimizar la latencia.

CDN usa una red geográfica distribuida de servidores web, así el contenido se entrega desde un punto cercano al usuario y no desde el servidor de origen.

Los nodos Edge son esa red geográfica distribuida de servidores web, ubicados en los límites de la red.

Y el enrutamiento Anycast redirige automáticamente la petición del usuario al servidor más cercano latencialmente.

Una CDN reduce la latencia actuando como una red distribuida de servidores que acercan los recursos al usuario final mediante dos componentes clave:

1. Nodos Edge (Servidores Perimetrales): Son servidores ubicados físicamente en los "límites" de la red (cerca de los Proveedores de Servicios de Internet o ISPs locales). En lugar de que la petición del usuario viaje miles de kilómetros hasta el servidor de origen en Alemania, el Nodo Edge local (por ejemplo, en Buenos Aires) intercepta la solicitud y entrega una copia guardada en su memoria/disco local (caché).

2. Enrutamiento Anycast: Es una técnica de enrutamiento BGP a nivel de red donde múltiples servidores geográficamente distintos comparten la misma dirección IP pública. Cuando el usuario realiza la petición HTTP/S, los routers de Internet dirigen los paquetes automáticamente al servidor que responda con la menor distancia de red / latencia (el nodo más cercano). Conclusión: Gracias al enrutamiento Anycast, el usuario es enviado al Nodo Edge más próximo sin percibir la complejidad de la infraestructura. Esto reduce los tiempos de ida y vuelta (Round-Trip Time o RTT), ahorra ancho de banda en la red WAN global y libera de carga al servidor de origen en Alemania.

Los servidores de borde funcionan bajo el principio de **Caché Bajo Demanda**:

1. **La primera vez que alguien pide algo:** Si eres la primera persona en Buenos Aires que pide ver una foto específica (`foto-producto.jpg`), el servidor de borde no la tiene. La pide a Alemania, te la entrega a ti y **guarda una copia temporal** en su memoria RAM o disco local.

2. **Las siguientes miles de personas:** Cuando otros usuarios en Argentina piden esa misma foto, el servidor de borde **ya tiene la copia local** y se la entrega de inmediato sin volver a viajar a Alemania.
3. **Contenido Dinámico:** Si intentas ver el saldo de tu cuenta bancaria o tu carrito de compras, eso **nunca se guarda en el borde** por razones de seguridad y porque cambia a cada segundo. Esas peticiones dinámicas el servidor de borde simplemente las "redirecciona" rápidamente al servidor central de origen.

Pregunta 7

Finalizado

Sin calificar

Imagina que debes diseñar la infraestructura web para una plataforma que espera recibir 50.000 conexiones simultáneas de usuarios para descargar recursos de forma masiva. ¿Qué servidor web (**Apache** o **Nginx**) elegirías para esta tarea y por qué? En tu respuesta, compara los modelos de arquitectura de ambos servidores (procesos/hilos vs. asíncrono/eventos) y analiza cómo influyen en el consumo de memoria RAM y la flexibilidad de configuración mediante archivos `.htaccess`.

Elegiría Nginx porque su modelo de eventos consume poca RAM y soporta alta concurrencia, mientras que Apache, aunque más flexible de configurar, usa un modelo de procesos/hilos que consume más memoria a medida que crecen las conexiones y no compensa el costo de memoria.

Apache HTTP Server: Arquitectura modular clásica basada en procesos/hilos. Configuración flexible (.htaccess).

Nginx: Arquitectura asíncrona guiada por eventos. Diseñado para alta concurrencia y bajo consumo de RAM.

Para manejar 50.000 conexiones simultáneas con alto rendimiento y bajo consumo de recursos, la elección adecuada es **Nginx**.

La justificación técnica basada en la comparación de ambas arquitecturas comprende los siguientes puntos:

1. **Modelo de Arquitectura y Concurrencia:**
    - **Nginx (Modelo Asíncrono orientado a Eventos):** Utiliza un número muy reducido de procesos (*Worker Processes*) ejecutados en un **bucle de eventos (*Event Loop*)**. Maneja miles de conexiones de forma no bloqueante dentro de un solo hilo, lo que le permite sostener una altísima concurrencia (superando el problema C10K) con un consumo de memoria RAM extremadamente bajo y casi constante.
    - **Apache (Modelo Clásico basado en Procesos/Hilos):** Tradicionalmente asigna un hilo o proceso por cada petición entrante (*Thread-Per-Request*). A medida que las conexiones simultáneas aumentan a decenas de miles, el consumo de memoria se dispara linealmente y el procesador pierde rendimiento debido al constante cambio de contexto (*context switching*) entre miles de hilos.
2. **Manejo de Memoria RAM:**
    - Crear un hilo en Apache puede consumir entre 2 MB y 4 MB de RAM por usuario conectado. Con 50.000 conexiones, la memoria requerida colapsaría el servidor. En Nginx, cada conexión inactiva o en espera consume apenas un par de kilobytes, permitiendo atender volúmenes masivos de tráfico con hardware modesto.
3. **Flexibilidad de Configuración (`.htaccess`):**
    - **Apache**: Ofrece una alta flexibilidad descentralizada al permitir que cada directorio tenga su propio archivo `.htaccess`. Esto permite a los desarrolladores cambiar reglas de reescritura sin reiniciar el servidor, pero implica que Apache deba analizar el disco buscando estos archivos en cada petición, reduciendo la velocidad de respuesta.
    - **Nginx:** No soporta archivos `.htaccess` por motivos de rendimiento. Todas sus directivas se compilan en un archivo de configuración centralizado en memoria RAM. Aunque esto reduce la flexibilidad

descentralizada para usuarios sin acceso de administrador, garantiza la máxima velocidad posible de E/S.

Conclusión:

Nginx es el estándar ideal para la capa frontal de alta concurrencia y entrega de contenido estático por su arquitectura de eventos eficiente, mientras que Apache se reserva para entornos donde la flexibilidad de configuración descentralizada por directorio sea un requisito indispensable.